Universidad Autónoma de Baja California

Campus Sauzal

Facultad de Ingeniería Arquitectura y Diseño

Ingeniería en Software y Tecnologías Emergentes

Laboratorio 1: Cifrado simetrico y asimetrico

Seguridad en Software

Grupo 971

Gerardo Yepez Tsuchiya 346176
Maria Fernanda Lopez Merino 373025
Roxana Wendoline Lambur Barrios 372864
Maria Fernanda Medrano Calderon 373143

Ensenada, B.C. 18 de Febrero de 2025

CIFRADO SIMÉTRICO

El **Cifrado César** es un método clásico de cifrado simétrico que funciona sustituyendo letras: cada letra del mensaje original se mueve un número fijo de posiciones en el alfabeto. Ese número es la clave, y para descifrar el mensaje se hace el proceso contrario usando la misma clave, por eso se considera un cifrado simétrico.

En el **Caesar Cipher con rotación variable**, el desplazamiento no se mantiene igual todo el tiempo, sino que cambia para cada letra del texto. Es decir, cada carácter se cifra con un movimiento diferente, según una clave que puede ser una lista de números o un patrón que se repite.

Esto hace que el cifrado sea más complicado que el César tradicional, porque ya no hay una sola rotación fija, sino varias. Aun así, sigue siendo un cifrado por sustitución, por lo que puede ser vulnerable a ataques modernos, sobre todo si el patrón de rotación es corto o fácil de predecir.

Implementación

- **Código fuente completo con comentarios** *(Se anexo el código en la entrega)*

- **Capturas de la aplicación funcionando**

```
ferme@chimki MINGW64 ~/OneDrive/Escritorio/SS
$  /usr/bin/env c:\\Users\\ferme\\AppData\\Local\\Programs\\Python\\Python313\\python.exe c:\\Users\\ferme\\.vscode\\extensions\\ms-python.d
ebugpy-2025.18.0-win32-x64\\bundled\\libs\\debugpy\\launcher 51216 -- c:\\Users\\ferme\\OneDrive\\Escritorio\\SS\\Cesar.py
=== Caesar Cipher con Rotación Variable ===

1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción:
```

- **Comparación con CyberChef (capturas)**
  "La seguridad del software es fundamental para proteger sistemas digitales contra amenazas cibernéticas"

Nuestro sistema

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar: La seguridad del software es fundamental para proteger sistemas digitales contra amenazas ciberneticas
Ingresa la clave (ej: 3 1 4): 3
Texto cifrado: Od vhjxulgdg gho vriwzduh hv ixqgdphqwdo sdud surwhjhu vlvwhpdv gljlwdohv frqwud dphqdcdv flehuqhwlfdv
```

CyberChef

Input

La seguridad del software es fundamental para proteger sistemas digitales contra amenazas cibernéticas

ABC 102 1 Raw Bytes LF

Output

Od vhjxulgdg gho vriwzduh hv ixqgdphqwdo sdud surwhjhu vlvwhpdv gljlwdohv frqwud dphqdcdv flehuqéwlfdv

Análisis de vulnerabilidades

- **Patrón repetitivo en la clave**, su **debilidad** sería si la clave es corta, la rotación se repite constantemente, un **atacante** puede detectar el patrón analizando el texto cifrado y encontrar la secuencia de rotaciones. Y su **mitigación** podría ser usando claves largas y aleatorias para reducir repetición.

- **Vulnerabilidad al análisis de frecuencia,** su **debilidad** sigue siendo un cifrado basado en sustitución, con suficiente texto cifrado, un **atacante** puede aplicar análisis estadístico para deducir rotaciones y reconstruir el texto original. Y su **mitigación** sería combinar el algoritmo con técnicas más complejas como transposición o cifrados modernos.

- **Ataque por fuerza bruta,** su **debilidad** es si la clave es pequeña o con rotaciones limitadas, se pueden probar combinaciones posibles, un **atacante** podría automatizar intentos de claves hasta encontrar un texto legible. Y su **mitigación** sería aumentar la longitud de clave, permitir rotaciones más amplias y aplicar claves no predecibles.

Casos de prueba

- **Texto normal con espacios y puntuación**

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar: Hola mundo, esto es una prueba.
Ingresa la clave (ej: 3 1 4): 2
Texto cifrado: Jqnc owpfq, guvq gu wpc rtwgdc.
```

- **Texto con caracteres especiales (ñ, acentos)**

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar: Mañana va a llover
Ingresa la clave (ej: 3 1 4): 4
Texto cifrado: Qesere ze e ppsziv
```

- **Mensaje muy corto (menos de 10 caracteres)**

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar: Hola
Ingresa la clave (ej: 3 1 4): 2
Texto cifrado: Jqnc
```

- **Mensaje largo (más de 500 caracteres)**

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar: Este es un mensaje largo diseñado para probar el funcionamiento del algoritmo Caesar Cipher con rotación variable. La intención de este caso es comprobar
que el programa puede cifrar y descifrar textos extensos sin perder información ni alterar los caracteres especiales como espacios, comas y puntos. Además, permite verificar que la
clave se repite correctamente cuando el texto es más largo que la longitud de la clave, asegurando que el proceso de cifrado se mantenga estable durante toda la ejecución del progra
ma.
Ingresa la clave (ej: 3 1 4): 3
Texto cifrado: Hvwh hv xq phqvdmh odujr glvhrdgr sdud suredu ho ixqflrqdplhqwr gho dojrulwpr Fdhvdu Flskhu frq urwdfltq yduldeoh. Od lqwhqfltq gh hvwh fdvr hv frpsuredu txh ho surju
dpd sxhgh fliudu b ghvfliudu whawrv hawhqvrv vlq shughu lqirupdfltq ql dowhudu orv fdudfwhuhv hvshfldohv frpr hvsdflrv, frpdv b sxqwrv. Dghpbv, shuplwh yhulilfdu txh od fodyh vh uhs
lwh fruuhfwdphqwh fxdqgr ho whawr hv pbv odujr txh od orqjlwxg gh od fodyh, dvhjxudqgr txh ho surfhvr gh fliudgr vh pdqwhqjd hvwdeoh gxudqwh wrgd od hmhfxfltq gho surjudpd.
```

- **Clave inválida o vacía**

```
1. Cifrar texto
2. Descifrar texto
3. Generar clave aleatoria
4. Salir
Selecciona una opción: 1
Ingresa el texto a cifrar:
Ingresa la clave (ej: 3 1 4): 2
Texto cifrado:
```

CIFRADO ASIMÉTRICO

Proceso de generación de llaves

Se utilizó **OpenSSL** versión **3.4.0** para la generación de un par de llaves **RSA** de **2048 bits** mediante el comando **genpkey**. Posteriormente se extrajo la llave pública con **pkey -pubout**. Se verificó el correcto funcionamiento cifrando y descifrando un mensaje de prueba usando **pkeyutl**.

Se generó la llave privada

```
C:\Users\wendy>openssl version
OpenSSL 3.4.0 22 Oct 2024 (Library: OpenSSL 3.4.0 22 Oct 2024)

C:\Users\wendy>openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
.+.......+......+......++++++++++++++++++++++++++++++++++++++++++*...+...+..............+.+.....+.....
.+.....+.........+...................+.....+.+.........+..+.......++++++++++++++++++++++++++++++++++++
....+....+......+..............+......+.......+.....+......+.......+...+............+.........+....
...+.....+.+....................+....+..++++++
...+......+.....+......++++++++++++++++++++++++++++++++++++++++++*.+...+..+...+.+...+..+..........+...
.............+.++++++++++++++++++++++++++++++++++++++++++*.....+...+....+...+...+..+..........+..+.+..
+....+...+.....+...+.............+......+...+.....+.............+...+.........+..+................
+.........+...+...+...+.+........+............+.........+.......+.........+........+......+....+...
.....+........+...+....+...+.....+.+.........+.....+............+.+...+..+.........+......+...+....
+...+..+.......+..................+........+....+..+...+.......+.....+...+.............+..+........
....+......+.....+..........+........+...+...........+........+.+......+.........+................
..+..........+..+....+......+......+.........+..............+.+.....+....+..+..................+..
...+...+....+.........+......+.........+............+...+.....+..........+......+......+..+...+....
..............+.....+......+....+...+..+...+.......+...........+.......+..+...+..........+..+...+..
.....................+....+............+...+.....+.+.....+...+.+......+........+.+..+.............
....................+...+.............+.....+..........+...+........+......+....+.....+....+......
+.......+..+...+.......+......+.....+.+...........+....+.....+....+.....+.+...........+...+........
..........+..............+....+..............+.+.....+.+.....+.+.........+.........+......+...+++++

C:\Users\wendy>dir private_key.pem
 El volumen de la unidad C es Windows-SSD
 El número de serie del volumen es: B0AD-3537

 Directorio de C:\Users\wendy

18/02/2026  11:04 p. m.             1,732 private_key.pem
               1 archivos          1,732 bytes
               0 dirs  234,195,439,616 bytes libres
```

Se generó la llave pública

```
C:\Users\wendy>openssl pkey -in private_key.pem -pubout -out public_key.pem

C:\Users\wendy>dir public_key.pem
 El volumen de la unidad C es Windows-SSD
 El número de serie del volumen es: B0AD-3537

 Directorio de C:\Users\wendy

18/02/2026  11:09 p. m.               460 public_key.pem
               1 archivos            460 bytes
               0 dirs  234,199,711,744 bytes libres
```

Se creó el mensaje para después cifrarlo con nuestra llave pública (solo para probar que funcionara)

```
C:\Users\wendy>echo Equipo 1 - Mensaje de prueba RSA > mensaje.txt

C:\Users\wendy>openssl pkeyutl -encrypt -pubin -inkey public_key.pem -in men
saje.txt -out mensaje_cifrado.bin

C:\Users\wendy>dir mensaje_cifrado.bin
 El volumen de la unidad C es Windows-SSD
 El número de serie del volumen es: B0AD-3537

 Directorio de C:\Users\wendy

18/02/2026  11:13 p. m.               256 mensaje_cifrado.bin
               1 archivos            256 bytes
               0 dirs  234,189,033,472 bytes libres
```

Aquí desciframos el mensaje de prueba con la llave privada

```
C:\Users\wendy>openssl pkeyutl -decrypt -inkey private_key.pem -in mensaje_c
ifrado.bin -out mensaje_descifrado.txt

C:\Users\wendy>type mensaje_descifrado.txt
Equipo 1 - Mensaje de prueba RSA
```

Herramientas utilizadas

OpenSSL es una herramienta muy confiable y bastante usada en el área de criptografía, ya que es considerada un estándar en muchos sistemas. Una de sus principales ventajas es que permite generar llaves RSA de forma sencilla y segura, además de que sus comandos son rápidos y directos.

Sin embargo, no es una herramienta muy amigable para personas que apenas están empezando, porque todo se maneja desde la consola con comandos. Esto puede causar errores si se escriben mal los parámetros o se confunden opciones, y también puede volverse un poco confuso el manejo de archivos si no se tiene buena organización.

En general, OpenSSL es una herramienta muy eficiente, pero se necesita cuidado y atención al usarla para evitar fallos al ejecutar los comandos.

Comparación: ¿Cuándo usarías cifrado simétrico vs asimétrico?

El cifrado **simétrico** se usa cuando se necesita proteger grandes cantidades de información de manera rápida, ya que tanto el emisor como el receptor utilizan la misma clave secreta para cifrar y descifrar. Un ejemplo muy común es AES, que se utiliza para proteger archivos, bases de datos y comunicaciones.

Por otro lado, el cifrado **asimétrico** se usa cuando se quiere intercambiar información de forma segura sin necesidad de compartir una clave previamente. Este tipo de cifrado utiliza una clave pública y una clave privada, y también sirve para autenticación y firmas digitales. Un ejemplo es RSA, que se usa principalmente para cifrar mensajes cortos o para intercambiar claves.

En resumen, el cifrado simétrico es más rápido pero requiere compartir la clave, mientras que el asimétrico es más seguro para iniciar el intercambio, aunque es más lento.

REFLEXIÓN GRUPAL

Lecciones aprendidas

Aprendimos que el cifrado César con rotación variable es más complicado porque cada letra se desplaza diferente. También entendimos mejor la diferencia entre cifrado simétrico y asimétrico. Además, vimos que OpenSSL es muy útil, pero requiere cuidado al usar comandos y archivos.

Dificultades encontradas

Tuvimos problemas al aplicar el patrón de rotación correctamente, pero se resolvió organizando bien la clave. También hubo dificultades con caracteres especiales como ñ y acentos. En RSA, lo más complicado fue manejar correctamente las llaves y archivos, lo cual se solucionó con orden y pruebas.

Aplicaciones prácticas

El cifrado simétrico se usa para proteger información rápidamente en archivos, discos y bases de datos. El asimétrico se usa en HTTPS, intercambio seguro de claves y firmas digitales. En la práctica se combinan: el asimétrico inicia la conexión y el simétrico cifra los datos.

Referencias

- *Caesar cipher*. (s/f). HackerRank. Recuperado de https://www.hackerrank.com/challenges/caesar-cipher-1/problem
- *Caesar cipher in cryptography.* (2016, junio 2). GeeksforGeeks. https://www.geeksforgeeks.org/ethical-hacking/caesar-cipher-in-cryptography/
- *OpenSSL*. (s/f). Openssl.org. Recuperado de https://openssl.org/
- *What is OpenSSL?* (s/f). F5, Inc. Recuperado de https://www.f5.com/glossary/openssl